CS543 WEB SERVICES

ASSIGNMENT 2 OF 14

Campus / Student Lost & Found Management System

Service Boundaries, Contracts & Database Schema

Design benchmark for reporting, matching, claiming, verifying, and returning lost belongings across a university campus.

Prepared as a technical design submission

1. Project overview

The Campus/Student Lost & Found Management System provides a common place for students and staff to report lost and found belongings, search reports, identify possible matches, submit ownership claims, review evidence, and coordinate the return of recovered items.

Assumptions

Users are students, staff, and authorized administrators. Reports include description, category, date, and campus location. A match is only a suggestion, not proof of ownership. Authorized reviewers decide claims. Proof details are private. Authentication and HTTPS are supplied by the application platform; passwords are stored as secure hashes.

Architecture rules

Each service owns its tables and controls writes to them. Cross-service identifiers are references exchanged through APIs, not direct cross-service foreign keys. The unified SQL is for assignment review; production should isolate service-owned data in separate databases or schemas.

2. Task 1 — Capabilities

#	Capability	Description	Service
1	Register lost item	Record item description, category, date, and last-seen location.	Item Report
2	Register found item	Record an item found on campus and its location.	Item Report
3	Search reports	Filter reports by keywords, category, location, type, and status.	Item Report
4	Suggest matches	Compare lost/found reports and store candidate matches with score/explanation.	Matching
5	Submit ownership claim	Submit proof details for a found item.	Claims & Verification
6	Review claim	Approve, reject, or request more information.	Claims & Verification
7	Send notifications	Notify relevant users of matches and claim/handover changes.	Notification
8	Coordinate handover	Arrange and confirm an in-person return after approval.	Notification
9	Manage profile/identity	Maintain profile details and campus identity attributes.	User

3. Task 2 — Service design

See services.drawio and services.png for the architecture benchmark. The following are proposed implementation roles, not actual named team assignments.

Service	Owned data	Responsibility	Proposed role
User Service	users, profiles	Profiles and campus identity.	Identity/Profile Backend Developer
Item Report Service	categories, lost_items, found_items	Authoritative lost/found reports, categories, locations, and report status.	Item-Management Backend Developer
Matching Service	matches	Candidate match records; reports remain owned by Item Report Service.	Search/Matching Backend Developer
Claims & Verification Service	claims, claim_verification	Claims, proof/review records, and claim lifecycle.	Claims/Verification Backend Developer
Notification Service	notifications, handovers	In-app notifications and handover records.	Notifications/Handover Backend Developer

Cross-service calls

Caller	Callee	Operations
Client/Web App	User Service	registerUser / getUserProfile
Client/Web App	Item Report Service	reportLostItem / reportFoundItem / searchItems
Item Report Service	Matching Service	findMatches
Matching Service	Item Report Service	getItemReport
Client/Web App	Claims & Verification Service	submitClaim / getClaim
Claims & Verification Service	Item Report Service	getItemReport / updateItemStatus

Caller	Callee	Operations
Claims & Verification Service	User Service	getUserProfile
Claims & Verification Service	Notification Service	sendNotification
Client/Web App	Notification Service	getNotifications / scheduleHandover
Notification Service	Claims & Verification Service	getClaim

Workflow

Item Report Service stores authoritative reports and requests candidate matches from Matching Service. Matching Service retrieves report details only through Item Report Service contracts. A claimant submits a claim to Claims & Verification Service, which checks item availability through Item Report Service, retrieves necessary public user information through User Service, records the claim in its own data store, and asks Notification Service to notify relevant parties. Approved claims can proceed to a scheduled handover.

4. Task 3 — Service contracts

Contracts define externally visible behavior. They hide table names, SQL, database ID formats, and internal implementation details. The logical API may be implemented using JSON over HTTPS.

User Service

Owns: users, profiles. **Responsibility:** Identity/Profile Backend Developer

Operation	Input	Output	Errors
registerUser	name, email, password, campusId?	userId, profile	EMAIL_EXISTS, INVALID_INPUT
getUserProfile	userId	publicProfile	USER_NOT_FOUND
updateProfile	userId, profileFields	updatedProfile	USER_NOT_FOUND, INVALID_INPUT
verifyCampusIdentity	userId, campusId	verificationStatus	USER_NOT_FOUND, IDENTITY_MISMATCH

Item Report Service

Owns: categories, lost_items, found_items. **Responsibility:** Item-Management Backend Developer

Operation	Input	Output	Errors
reportLostItem	reporterId, title, description, categoryId, location, lostDate	lostReportId, status	INVALID_INPUT, CATEGORY_NOT_FOUND
reportFoundItem	reporterId, title, description, categoryId, location, foundDate	foundReportId, status	INVALID_INPUT, CATEGORY_NOT_FOUND
getItemReport	itemType, itemId	itemReport	ITEM_NOT_FOUND
searchItems	query?, categoryId?, location?, itemType?, status?	items[], pageInfo	INVALID_FILTER, SERVICE_UNAVAILABLE
updateItemStatus	itemType, itemId, expectedStatus, newStatus	updatedStatus	ITEM_NOT_FOUND, INVALID_TRANSITION, CONFLICT

Matching Service

Owns: matches. **Responsibility:** Search/Matching Backend Developer

Operation	Input	Output	Errors
findMatches	itemType, itemId	matches[] with score and explanation	ITEM_NOT_FOUND, MATCHING_UNAVAILABLE
getMatch	matchId	matchDetails	MATCH_NOT_FOUND
updateMatchStatus	matchId, status	updatedMatch	MATCH_NOT_FOUND, INVALID_STATUS

Claims & Verification Service

Owns: claims, claim_verification. **Responsibility:** Claims/Verification Backend Developer

Operation	Input	Output	Errors
submitClaim	claimantId, foundItemId, proofDetails, idempotencyKey	claimId, status	ITEM_NOT_FOUND, ITEM_UNAVAILABLE, DUPLICATE_ACTIVE_CLAIM, INVALID_INPUT
getClaim	claimId	claimDetails	CLAIM_NOT_FOUND, FORBIDDEN
reviewClaim	claimId, reviewerId, decision, notes	claimStatus, decision	CLAIM_NOT_FOUND, FORBIDDEN, INVALID_TRANSITION
updateClaimStatus	claimId, expectedStatus, newStatus	updatedStatus	CLAIM_NOT_FOUND, INVALID_TRANSITION, CONFLICT

Notification Service

Owns: notifications, handovers. **Responsibility:** Notifications/Handover Backend Developer

Operation	Input	Output	Errors
sendNotification	recipientId, type, message, relatedEntityRef	notificationId, deliveryStatus	USER_NOT_FOUND, DELIVERY_FAILED

Operation	Input	Output	Errors
getNotifications	userId, unreadOnly?, page?	notifications[], pageInfo	USER_NOT_FOUND
markAsRead	userId, notificationId	readStatus	NOTIFICATION_NOT_FOUND, FORBIDDEN
scheduleHandover	claimId, participants, location, scheduledAt	handoverId, status	CLAIM_NOT_APPROVED, INVALID_TIME
confirmHandover	handoverId, confirmerId	handoverStatus	HANDOVER_NOT_FOUND, FORBIDDEN

5. Task 4 — Full specification: submitClaim

Purpose

Creates an ownership claim for a found item. It returns a claim reference and status; it does not prove ownership or guarantee approval.

Public contract

Element	Specification
Operation	submitClaim
Input	claimantId; foundItemId; proofDetails; idempotencyKey. In production, claimant identity must be derived from the authenticated principal, not trusted solely from client input.
Success output	claimId; status=SUBMITTED; submittedAt; nextStep=REVIEW_PENDING.
Errors	INVALID_INPUT; UNAUTHENTICATED; FORBIDDEN; ITEM_NOT_FOUND; ITEM_UNAVAILABLE; DUPLICATE_ACTIVE_CLAIM; IDEMPOTENCY_KEY_REUSED; DEPENDENCY_UNAVAILABLE; INTERNAL_ERROR.
Idempotency	Equivalent retry with the same claimant and key returns the existing result. Reusing the key with a different payload returns IDEMPOTENCY_KEY_REUSED.

Preconditions and validation

The caller must be authenticated; required fields must be present and within size limits. proofDetails must not be empty and is sensitive. The found item must exist and be eligible for claiming. A claimant cannot have another active claim for the same item. Only authorized reviewers may decide claims.

Processing steps

Step	Action
1	Authenticate caller and derive claimant identity from trusted security context.
2	Validate fields, proofDetails length, idempotency key, and request schema.
3	Check idempotency: return the prior result for equivalent retries; reject conflicting key reuse.
4	Call Item Report Service getItemReport and confirm the found item is AVAILABLE. Never query its database directly.
5	Check for an active claim by the same claimant/item; enforce the invariant with a database constraint where possible.
6	Create the claim in Claims & Verification Service's own database with status SUBMITTED.
7	Call Item Report Service updateItemStatus with expectedStatus=AVAILABLE and newStatus=CLAIM_PENDING.
8	Call Notification Service sendNotification to alert relevant users that a claim awaits review.
9	Return claimId, SUBMITTED status, timestamp, and review-pending next step; do not expose proof details to unrelated users.

Failures and consistency

If item lookup fails, do not create the claim; return ITEM_NOT_FOUND or DEPENDENCY_UNAVAILABLE. If claim persistence fails, do not report success. If item status update fails after claim persistence, use a compensating workflow or reconciliation queue; do not claim a distributed atomic transaction. If notification delivery fails, keep the valid claim and retry notification asynchronously. Reconcile claims stuck in SUBMITTED while the item remains AVAILABLE, and items marked CLAIM_PENDING without an active claim. Use bounded retries with backoff.

Privacy and caller-visible details

Only the claimant and authorized reviewers may access proof details. Avoid logging full proof data; audit reviewer decisions and enforce role-based authorization. Callers see the operation, inputs, output, status, and documented errors, not SQL, tables, internal retry queues, or database implementation.

6. Task 5 — Database schema

See schema.drawio and schema.png for the ER diagram and schema.sql for PostgreSQL 14+ DDL.

Service	Tables	Relationship notes
User Service	users, profiles	profiles.user_id is local to User Service.
Item Report Service	categories, lost_items, found_items	Item reports reference categories locally. reporter_id is an external User Service identifier.
Matching Service	matches	lost_item_id and found_item_id are external IDs; Item Report Service remains authoritative.
Claims & Verification Service	claims, claim_verification	claim_verification.claim_id is local; claimant_id, found_item_id, verified_by are external identifiers.
Notification Service	notifications, handovers	recipient_id, claim_id, and user references are external identifiers.

Schema decisions

Local records use generated BIGINT primary keys. Email and campus ID are unique where present. Statuses are constrained to defined values. Match scores are constrained to 0–100. Claims include idempotency keys and a partial unique index to reduce duplicate active claims. Notification delivery state is tracked separately from claim status. Handover time must be later than record creation.

Production note

The single SQL file is a review sketch. In production, separate service databases/schemas and credentials enforce ownership. Cross-service IDs are validated through APIs and reconciliation rather than cross-service joins.

7. Task 6 — Five service properties

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled
User	PASS: API endpoint	PASS: owns profile data	PASS: operations documented	PASS: local ownership	PASS: consumers use APIs
Item Report	PASS: API endpoint	PASS: owns reports/categories	PASS: operations documented	PASS: authoritative owner	PASS: callers use contracts
Matching	PASS: API endpoint	PASS: owns match results	PASS: operations documented	PASS: separately scalable	PASS: report details via Item API
Claims & Verification	PASS: API endpoint	PASS: owns claims/reviews	PASS: operations documented	PASS: local claim lifecycle	PASS: item/user/notification via APIs
Notification	PASS: API endpoint	PASS: owns notifications/handovers	PASS: operations documented	PASS: local delivery data	PASS: public contracts

Validation caveat and fixes

These are design-level assessments; deployed reachability and runtime independence cannot be proven from diagrams alone. Reachability requires deployed endpoints and service discovery. Self-containment requires separate data ownership and credentials. Independence requires independently configurable/deployable services. Loose coupling requires versioned contracts, timeouts, bounded retries, idempotency, and asynchronous notification/reconciliation. Direct cross-service SQL or shared write tables would violate the design and must be removed.

8. Conclusion

The design divides the Lost & Found workflow into five services with clear responsibilities, explicit contracts, and single-service table ownership. The submitClaim specification addresses validation, idempotency, privacy, partial failure, and eventual reconciliation. The diagrams and SQL form a consistent design benchmark for implementation.